📘 **AI/ML Internship – Day 4 Notes & Tasks**

🟢 **Module 3: Python for AI**

📌 **Part 1: File Handling in Python (Detailed)**

🔹 **What is File Handling?**

File handling allows us to:

- Store data permanently
- Read data later
- Work with large datasets (important for AI)

👉 Example:

- Saving student records
- Reading dataset files

🔹 **File Modes Explained**

Mode	Meaning	Use Case
r	Read	Read existing file
w	Write	Create/overwrite file
a	Append	Add data without deleting
x	Create	Create new file only

🔹 **Writing to a File**

```
file = open("data.txt", "w")
file.write("Hello Intern\n")
file.write("Welcome to AI ML")
file.close()
```

👉 Notes:

- "w" will **overwrite** existing content
- Use \n for new line

🔹 Reading from a File

```
file = open("data.txt", "r")
content = file.read()
print(content)
file.close()
```

🔹 Read Line by Line

```
file = open("data.txt", "r")

for line in file:
    print(line)

file.close()
```

👉 Useful for large files (datasets)

🔹 Append to File

```
file = open("data.txt", "a")
file.write("\nNew Student Added")
file.close()
```

🔹 Best Practice (with statement)

```
with open("data.txt", "r") as file:
    content = file.read()
    print(content)
```

👉 Advantage:

- Automatically closes file
- Safer and cleaner

📌 Part 2: Exception Handling (Detailed)

🔹 What is an Exception?

An exception is an error that occurs during program execution.

👉 Example:

- Dividing by zero
- Invalid input

🔹 **Why Use Exception Handling?**

- Prevent program crash
- Handle errors properly
- Improve user experience

🔹 **Basic Try-Except**

```
try:
    x = int(input("Enter number: "))
    print(10 / x)
except:
    print("Something went wrong")
```

🔹 **Specific Exceptions**

```
try:
    x = int(input("Enter number: "))
    print(10 / x)
except ZeroDivisionError:
    print("Cannot divide by zero")
except ValueError:
    print("Please enter valid number")
```

👉 Always prefer **specific exceptions**

🔹 **Else Block**

```
try:
    x = int(input())
except:
    print("Error")
```

```
else:
    print("No error occurred")
```

👉 Runs only if no error occurs

🔹 **Finally Block**

```
try:
    print("Running")
except:
    print("Error")
finally:
    print("This always runs")
```

👉 Used for:

- Closing files
- Cleaning resources

🔹 **Combining File Handling + Exception**

```
try:
    with open("data.txt", "r") as file:
        print(file.read())
except FileNotFoundError:
    print("File not found")
```

📌 **Part 3: Real-World Usage (Very Important)**

👉 In AI projects:

- Data is stored in files (CSV, TXT)
- Errors may occur while reading data

👉 So we use:

- File handling → to manage data
- Exception handling → to avoid crashes

📌 **Part 4: Advanced Practice Concepts**

🔹 Count Lines in File

```
with open("data.txt", "r") as file:
    lines = file.readlines()
    print(len(lines))
```

🔹 Count Words in File

```
with open("data.txt", "r") as file:
    text = file.read()
    words = text.split()
    print(len(words))
```

🔹 Write Multiple Inputs

```
with open("data.txt", "a") as file:
    for i in range(3):
        name = input("Enter name: ")
        file.write(name + "\n")
```

🎯 Tasks for Day 4

🧠 Task 1: Theory (Detailed Answers)

1. Explain file handling with examples
2. Difference between read, write, append
3. What is exception handling?
4. Why use try-except?
5. Explain finally block with example

💻 Task 2: File Handling Practice

1. Create file and write your details
2. Read file and display
3. Append new data

4. Count number of lines
5. Count number of words

💻 Task 3: Exception Handling Practice

1. Handle divide by zero
2. Handle invalid input
3. Use try-except-else-finally
4. Handle file not found error

🔥 Task 4: Logic Challenge

1. Find longest word in file
2. Count vowels in file
3. Remove duplicate lines